

Chức năng vai trò và API Plant Disease

Bộ tài liệu 2 trong 3 • Ngày 07 tháng 09 năm 2026 • Phiên bản đề xuất 1

1 Mô hình quyền của hệ thống

Hệ thống có bốn role trong database: user, manager, technician và admin. Guest là người chưa đăng nhập. Managed User là user được Manager tạo và có thể được gán vào Farm. Quan hệ cấp tài khoản không tự cấp quyền đọc dữ liệu riêng của người được tạo.

Admin tạo Manager hoặc Technician. Manager tạo Managed User và sở hữu Farm. Farm có nhiều Managed User; một Managed User có thể được gán vào nhiều Farm của cùng Manager theo dữ liệu hiện tại. Mỗi Scan có đúng một chủ tài khoản và có thể gán một Farm. Manager nhận thống kê Farm; quyền đọc Scan và ảnh cá nhân vẫn theo chủ Scan.

Ma trận dưới đây là quyền mục tiêu của bản 8 tuần. Một số chức năng chưa có API, được ghi ĐỀ XUẤT ở các trang sau. Dấu Có không bỏ qua kiểm tra đối tượng và trạng thái tài khoản.

Chức năng	Guest	User	Manager	Technician	Admin
Tra cứu bệnh	Có	Có	Có	Có	Có
Predict	Basic	Tối đa Standard	Tối đa Standard	Tối đa Advanced	Tối đa Advanced
Scan và ảnh cá nhân	Không	Của mình	Của mình	Của mình	Của mình
Gắn Scan vào Farm	Không	Có membership	Sở hữu Farm	Không	Không
Farm CRUD	Không	Không	Sở hữu Farm	Không	Không
Tạo Managed User và gán Member	Không	Không	User do mình tạo	Không	Không
Tạo Technician hoặc Manager	Không	Không	Không	Không	Có
Grad-CAM	Không	Không	Scan của mình	Scan của mình	Không
Gửi đề xuất nội dung	Không	Không	Không	Có	Không
Duyệt và sửa nội dung	Không	Không	Không	Không	Có
Dashboard Farm	Không	Không	Farm sở hữu	Không	Không
Giám sát toàn hệ thống	Không	Không	Không	Không	API Admin

User trong bảng bao gồm cả User độc lập và Managed User. Có quyền Advanced Predict không đồng nghĩa có quyền Grad-CAM; đây là hai nghiệp vụ khác nhau. Nếu sau này Technician hỗ trợ Farm hoặc Manager xem ảnh của thành viên, cần quan hệ phân công hoặc namespace Farm riêng; chưa cấp quyền này trong bản MVP.

2 Hợp đồng API và Auth hiện có

Tất cả đường dẫn nghiệp vụ dưới đây có tiền tố `/api/v1`. HIỆN CÓ là code đang triển khai. ĐỀ XUẤT là hợp đồng để phát triển, chưa được coi là endpoint hoạt động. Hiện có 27 endpoint nghiệp vụ; GET / là thông báo chạy ứng dụng và không tính vào danh sách đó.

API bảo vệ dùng Authorization Bearer access_token. Login nhận JSON. Refresh đọc HttpOnly Cookie, không nhận refresh token qua Bearer. Login không tự khiến mọi request có access token; FE phải gắn header cho API cần đăng nhập.

Method	Endpoint	Quyền	Hành vi và kết quả
POST	/auth/register	Public	HIỆN CÓ; role=user; username, email, password, full_name; 201
POST	/auth/login	Public	HIỆN CÓ; JSON username/password; access token 15 phút và cookie refresh 7 ngày; 200
POST	/auth/refresh	Cookie	HIỆN CÓ; so token_version và cấp cookie mới; 200 hoặc 401
GET	/auth/me	Đăng nhập	HIỆN CÓ; thông tin tài khoản an toàn; không password_hash; 200
POST	/auth/logout	Đăng nhập	HIỆN CÓ; token_version tăng và xóa cookie; 204
POST	/auth/change-password	Chính mình	ĐỀ XUẤT Tuần 5; current_password, new_password; hash mới và revoke trong cùng transaction; 204
PATCH	/admin/users/{user_id}/status	Admin	ĐỀ XUẤT Tuần 5; status active hoặc suspended và reason; 200

Quy trình đổi mật khẩu và khóa tài khoản

Đổi mật khẩu phải xác minh mật khẩu cũ, kiểm độ mạnh, lưu hash mới, tăng token_version bằng phép tăng an toàn trong database và commit một lần. Sau đó xóa cookie và yêu cầu đăng nhập lại; không trả password hoặc hash. Mật khẩu User PostgreSQL là cấu hình hạ tầng, không thay đổi qua endpoint này.

Admin khóa tài khoản khi có lý do; cả login, refresh và dependency đọc access token đều phải kiểm status. Không cho khóa chính Admin đang thao tác hoặc Admin active cuối cùng; kiểm điều kiện cạnh tranh trong transaction. Mở lại tài khoản không phục hồi token đã thu hồi. Manager chưa có quyền khóa User trong bản MVP.

Hành vi bảo mật cần giữ

Register và ManagerCreateUserRequest từ chối role truyền từ client. Service dùng chung cũng cần xác minh cặp actor và role đích. Refresh hiện tạo token mới nhưng chưa phát hiện phát lại token cũ theo token family. Nếu chưa mở rộng cơ chế này, mô tả là cấp lại refresh cookie và thu hồi bằng token_version.

3 Predict và hợp đồng kết quả

Method	Endpoint	Quyền	Hành vi và kết quả
GET	/predict/capabilities	Public hoặc token	HIỆN CÓ; role, default_mode, allowed_modes, models_by_mode và policy_version; 200
POST	/predict	Public hoặc token	HIỆN CÓ; multipart file, mode tùy chọn, farm_id tùy chọn; 200

Mode	Model hiện được dùng	Điều kiện quyền
basic	EfficientNet-B0	Mọi actor
standard	EfficientNet-B0 và MobileNetV2	User, Manager, Technician, Admin
advanced	EfficientNet-B0, MobileNetV2, ResNet50	Technician, Admin

auto là lựa chọn đầu vào để server lấy mode mặc định; không phải mode lưu vào Scan mới. Client được chọn mode thấp hơn quyền tối đa. Guest hoặc User gửi mode vượt quyền nhận 403. Token đã gửi nhưng không hợp lệ nhận 401, không được tự chuyển sang Guest.

Backend nhận JPEG và PNG, kiểm MIME thực, giới hạn mặc định 10 MiB và 20 triệu pixel; trả 413 khi vượt giới hạn, 415 khi không hỗ trợ định dạng hoặc MIME không khớp, 422 khi ảnh hỏng. Quy trình inference chuyển RGB, resize trực tiếp 224×224, ToTensor và normalize ImageNet. Mỗi model dùng temperature riêng rồi average probabilities.

Kết quả được chấp nhận và bị từ chối

validation_status là nguồn để FE quyết định hiển thị chẩn đoán. Khi accepted, có label và có thể nội dung disease_info. Khi low_confidence hoặc ambiguous, label và treatment là null; Top-K giữ để phân tích và không trình bày như chẩn đoán chắc chắn. is_valid_leaf là field tương thích đang biểu diễn policy chấp nhận, không chứng minh có detector lá.

Các ngưỡng hiện tại là confidence dưới 0.3, margin dưới 0.05 hoặc JS divergence trên 0.3. ood_score là heuristic tổng hợp; energy là số chẩn đoán, không có ngưỡng đã được ML xác nhận. model_error tồn tại trong miền schema nhưng lỗi toàn bộ inference hiện thường trả 503, không có Scan hợp lệ mới.

Guest không lưu ảnh hoặc Scan. User đăng nhập lưu Scan, Top-K ensemble và kết quả từng model cùng transaction. Khi một model lỗi, Standard có thể còn một model; Advanced phải còn tối thiểu hai. Response đánh dấu degraded và liệt kê error_code. Phải giữ nguyên số model thực chạy và model_version_id trong lịch sử.

Với người đã đăng nhập, Farm assignment hiện theo best-effort: Farm không hợp lệ thì Scan lưu cá nhân, farm_id=null và warning. FE phải thông báo không gán được Farm. Riêng Guest gửi farm_id nhận 400 trước inference. Giữ hành vi này trong v1; thay đổi cần cập nhật test và xác nhận contract cùng FE.

4 Farm Members và quản lý tài khoản

Tất cả API trong hai bảng sau là HIỆN CÓ. Mỗi thao tác Farm kiểm Manager sở hữu. User được gán Farm vẫn không có quyền Farm CRUD. Admin không được dùng Farm API của Manager.

Method	Endpoint	Quyền	Hành vi và kết quả
POST	/farms	Manager	name, location_text; owner server gán; 201
GET	/farms	Manager	Chỉ Farm sở hữu và chưa archive; 200
GET	/farms/{farm_id}	Chủ Farm	Chi tiết Farm; 200
PUT	/farms/{farm_id}	Chủ Farm	Cập nhật field gửi lên; không đổi owner; 200
DELETE	/farms/{farm_id}	Chủ Farm	Gán archived_at; giữ liên kết lịch sử; 204
POST	/farms/{farm_id}/members	Chủ Farm	Body user_id; User do Manager này tạo; 201, trùng 409
GET	/farms/{farm_id}/members	Chủ Farm	Danh sách thành viên; 200
DELETE	/farms/{farm_id}/members/{user_id}	Chủ Farm	Xóa quan hệ phân công; giữ tài khoản và Scan; 204

Method	Endpoint	Quyền	Hành vi và kết quả
POST	/admin/users	Admin	Chỉ role technician hoặc manager; 201
GET	/admin/users	Admin	Danh sách có filter role; 200
POST	/manager/users	Manager	Không có role trong body; hardcode user và created_by; 201
GET	/manager/users	Manager	Chỉ user do mình tạo; 200

Bổ sung cần thiết cho màn chọn Farm

ĐỀ XUẤT Tuần 5: GET /me/farms cho tài khoản đăng nhập. Manager nhận Farm sở hữu còn hoạt động; User nhận Farm có membership hợp lệ và người tạo phù hợp; User độc lập, Technician hoặc Admin nhận danh sách rỗng. Chỉ trả id, name, location_text và thông tin đủ để chọn; không trả danh sách thành viên của Farm. Cần kiểm lại membership lúc lưu Scan, không tin danh sách FE đã tải trước đó.

Các list API ngoài scans hiện chưa có phân trang. Đề xuất limit tối đa 100, offset không âm và thứ tự ổn định. Giai đoạn chuyển tiếp cho FE chủ động gửi limit; chỉ bật mặc định 20 khi FE đã xử lý phân trang. Giữ dạng response list chưa đủ bảo đảm tương thích nếu âm thầm cắt danh sách. Envelope items/total là thay đổi riêng.

PUT hiện dùng cập nhật từng phần và giữ cách dùng này trong v1. HIỆU CHỈNH ĐỀ XUẤT, chưa triển khai: field thiếu giữ nguyên; null chỉ xóa field nullable; null ở field bắt buộc trả 422. Code hiện còn cho null đi tới database. Không đổi ngay sang PATCH trên các endpoint đã được FE dùng.

5 Nội dung bệnh và lịch sử Scan

Các API trong bảng này là HIỆN CÓ.

Method	Endpoint	Quyền	Hành vi và kết quả
GET	/disease-info	Public	List nội dung is_active=true; 200
GET	/disease-info/{label_key}	Public	Chi tiết nhãn đang public; 200
POST	/disease-info	Admin	Tạo mới hoặc khôi phục record đã ẩn; 201
PUT	/disease-info/{label_key}	Admin	Sửa disease_name, description, treatment, severity_level; 200
DELETE	/disease-info/{label_key}	Admin	is_active=false; giữ row; 204
GET	/scans/history	Chủ Scan	List Scan của chính mình; limit 1 đến 100 và offset; 200
GET	/scans/{scan_id}	Chủ Scan	Kết quả, disease hiện hành, Top-3 và từng model; 200
DELETE	/scans/{scan_id}	Chủ Scan	Xóa Scan và kết quả con; dọn file ảnh; 204

ĐỀ XUẤT Tuần 5: GET /scans/{scan_id}/image trả file ảnh sau kiểm tra chủ Scan, cùng quy tắc với detail. Dùng Cache-Control phù hợp dữ liệu riêng; FE dùng request có Authorization rồi tạo URL blob để hiển thị ảnh. Không đưa Bearer token vào query string và không public thư mục uploads.

Quy tắc nội dung

label_key là định danh machine learning ổn định, không đổi trong generic update. Code hiện cho Admin tạo label bất kỳ nếu hợp lệ schema; đề xuất kiểm label được hỗ trợ trước khi public. Một record nội dung mới không làm checkpoint có thêm output class. Nội dung của healthy nên nói cây khỏe mạnh và không hiển thị diễn giải như cần trị bệnh chỉ vì severity_level mặc định.

POST hiện có hành vi khôi phục record đã soft-delete và cập nhật nội dung; cần ghi rõ cho FE. Giữ tương thích trong release này. Duplicate đang active hiện nhận 400 ở một số flow; không ghi mọi xung đột hiện có đều là 409.

Quy tắc lịch sử

Scan không có generic PUT để sửa nhãn hoặc confidence sau inference. Admin, Manager và Technician dùng /scans chỉ cho Scan của chính họ. Admin xem toàn hệ thống qua /admin/scans ở trang sau. Xóa Scan làm số liệu dashboard hiện thời giảm; nếu cần thống kê bất biến phải bổ sung chính sách retention riêng, không giả định đã có.

Hiện scan detail đọc DiseaseInfo mới nhất kể cả nội dung đã inactive; do đó màn hình nên gọi là hướng dẫn hiện hành. Nếu cần đúng nội dung từng hiển thị lúc chẩn đoán, ghi disease_snapshot cho Scan mới. Không hồi tố snapshot cũ bằng nội dung mới mà gọi là lịch sử nguyên bản.

6 Đề xuất nội dung và quy trình duyệt

ĐỀ XUẤT Tuần 6. Bản 8 tuần tập trung sửa nội dung của nhãn đã được model hỗ trợ. Yêu cầu nhãn mới và retraining được đưa vào backlog riêng. Cách này giúp trạng thái duyệt nội dung không bị trộn với trạng thái sẵn sàng của model.

Method	Endpoint	Quyền	Hành vi và kết quả
POST	/disease-proposals	Technician	label_key, base_content_version và nội dung đề xuất; 201 pending
GET	/disease-proposals/mine	Technician	Chỉ đề xuất của mình; filter status, limit, offset; 200
GET	/admin/disease-proposals	Admin	Filter status, label_key, proposer_id và thời gian; 200
PUT	/admin/disease-proposals/{id}/approve	Admin	Duyệt pending với revision còn đúng; 200
PUT	/admin/disease-proposals/{id}/reject	Admin	review_note bắt buộc; 200

Giữ PUT approve và reject như hợp đồng roadmap trước. Lock proposal trước: đã ở đúng quyết định yêu cầu thì trả record hiện có, không ghi thêm hoặc kiểm lại revision; quyết định đối nghịch trả 409. Chỉ proposal pending mới đi tiếp vào bước kiểm tra và cập nhật.

Luồng gửi và duyệt

Technician đọc disease_info cùng content_version, nhập thay đổi rồi gửi proposal. Server xác nhận role và nhãn tồn tại, ghi người đề xuất từ token, status=pending. Technician xem kết quả và review_note qua mine; không sửa nội dung public trực tiếp.

Với approve còn pending, trong cùng transaction service lock nội dung đích, kiểm is_active=true và base_content_version khớp. Nội dung đã ẩn hoặc revision đổi trả 409. Sau đó áp dụng field cho phép, tăng content_version, cập nhật reviewer, reviewed_at, status và audit rồi commit một lần. Update, archive và restore Disease đều phải tăng content_version.

Reject chỉ ghi trạng thái reviewed và lý do, không thay disease_info. Sau khi reviewed, bản đề xuất bất biến; Technician muốn sửa gửi một proposal mới. Không có DELETE proposal ở MVP để bảo toàn dấu vết duyệt.

Điều chỉnh CRUD để bảo đảm transaction

CRUD hiện có commit trong một số hàm. Workflow approve không được gọi hàm tự commit nội dung rồi mới cập nhật proposal. Cần cung cấp hàm chỉ flush hoặc thực hiện write trong service cùng session transaction. Test giả lập lỗi giữa quá trình phải cho thấy cả nội dung lẫn trạng thái vẫn ở bản trước.

Đề xuất thêm content_version vào response Disease Info là thay đổi bổ sung field. FE dùng nó để gửi proposal; các màn chỉ đọc cũ tiếp tục hoạt động.

7 Giải thích và giám sát

Các endpoint dưới đây là ĐỀ XUẤT. Giải thích thuộc phần có thể cắt nếu ML hoặc lịch làm việc không cho phép hoàn thiện trong Tuần 6.

Method	Endpoint	Quyền	Hành vi và kết quả
POST	/scans/{scan_id}/explanations	Technician hoặc Manager là chủ Scan	Body model_version_id và target_label tùy chọn; tạo Grad-CAM hoặc dùng cache; 201 hoặc 200
GET	/scans/{scan_id}/explanations/{id}	Cùng quyền tạo	Trả ảnh heatmap đã có; không sinh lại; 200
GET	/stats/farm/{farm_id}	Manager sở hữu	Tuần 5; aggregate Scan đã gắn Farm; from, to; 200
GET	/stats/admin/overview	Admin	Tuần 5; tổng User, Farm, Scan và rejection; 200
GET	/stats/admin/recent-invalid	Admin	Tuần 5; list rejection gần nhất, phân trang; 200
GET	/admin/scans	Admin	Tuần 5; filter user_id, validation_status, is_valid_leaf, from, to; 200
GET	/admin/scans/{scan_id}/image	Admin	Tuần 5; ảnh để giám sát, có audit; 200

Giải thích phải gắn đúng model

model_version_id phải xuất hiện trong kết quả thành công của Scan. Dùng đúng checkpoint lịch sử, không dùng active version mới để giải thích kết quả cũ. Nếu bundle đã mất, trả 503 với error_code an toàn. target_label mặc định là top-1 của model được chọn; nếu client chọn nhãn khác, nhãn phải nằm trong classes của chính version đó và được lưu vào metadata.

POST trả id, model_version_id, target_label, method, algorithm_version và image_url có quyền. GET trả bytes ảnh của đúng explanation thuộc scan_id. Bản tối giản chạy đồng bộ có timeout, trạng thái succeeded hoặc failed; chưa cần queue worker. Không quảng bá heatmap là vùng tổn thương đã được segment hoặc là giải thích chung của cả ensemble.

Thống kê phải có mẫu số rõ ràng

Tổng Scan gồm accepted và rejected. Tỷ lệ rejected bằng số rejected chia tổng Scan trong khoảng lọc; dữ liệu legacy tách riêng nếu thiếu status đáng tin cậy. Phân bố bệnh chỉ tính Scan accepted, mỗi Scan đóng góp một nhãn ensemble, không cộng cả ba model thành ba lượt chẩn đoán.

Dùng khoảng thời gian UTC [from, to), to lớn hơn from và mặc định khoảng hữu hạn. Manager dashboard chỉ trả aggregate, không tự cấp quyền xem ảnh cá nhân thành viên. Scan gắn Farm trước khi User bị gỡ membership vẫn thuộc lịch sử Farm. Snapshot sự thật lúc tạo không được suy ra từ membership hiện tại.

8 Phiên bản model và cấu hình mode

Method	Endpoint	Quyền	Hành vi và kết quả
GET	/model-versions	Admin	ĐỀ XUẤT Tuần 6; filter model_type, active, enabled, phân trang; 200
POST	/model-versions	Admin	ĐỀ XUẤT Tuần 6; đăng ký bundle đã được đặt trong vùng artifact cho phép; 201
PUT	/model-versions/{id}/activate	Admin	ĐỀ XUẤT Tuần 6; kiểm tra, warm-up và active cùng type; 200
GET	/admin/inference-policies	Admin	TÙY CHỌN P2; list mode và các revision; 200
POST	/admin/inference-policies	Admin	TÙY CHỌN P2; tạo revision mode mới với model_types, weights, thresholds; 201
PUT	/admin/inference-policies/{id}/activate	Admin	TÙY CHỌN P2; xác minh cấu hình và đổi policy active cùng mode; 200

Hai thao tác quản trị khác nhau

Model Version chọn trọng số đang dùng cho một kiến trúc. Ví dụ activate EfficientNet v2 thay EfficientNet v1, còn MobileNet và ResNet giữ nguyên. Database đã có ràng buộc tối đa một version active trên từng model_type; API activate chưa có.

Inference Policy chọn kiến trúc nào tham gia Basic, Standard hoặc Advanced, trọng số và rejection thresholds. Ví dụ đổi Standard từ EfficientNet cộng MobileNet sang EfficientNet cộng ResNet. Code hiện chưa có bảng hoặc API policy; không thể thực hiện chỉ bằng đổi is_active trong model_versions.

Nếu triển khai policy P2, quyền role đối với mode vẫn giữ cố định. Admin được cấu hình 1 đến 3 kiến trúc đã hỗ trợ trong mỗi mode; tổng trọng số phải hợp lệ, class order và preprocessing tương thích, model đang enabled và có active version. Các default 1, 2, 3 model là cấu hình ban đầu, không phải bảo đảm mọi revision luôn có đúng số đó. Min-success và degraded policy phải lưu theo revision thay vì suy từ tên mode.

Quy trình đăng ký và activate

Artifact deployment diễn ra trước register. Body chỉ nhận khóa bundle hoặc đường dẫn tương đối được kiểm soát, không nhận URL tải tùy ý hoặc pickle không tin cậy. Service kiểm SHA 256, manifest, kiến trúc, output, classes, temperature hữu hạn dương và preprocessing. Không đưa secret hoặc path máy chủ đầy đủ vào response lỗi.

Warm-up trước khi đổi trạng thái; transaction serialize theo model_type rồi tắt version cũ và bật version mới. Request đang xử lý dùng các ID đã chốt lúc bắt đầu. Version cũ có Scan tham chiếu phải được giữ; rollback là activate lại version cũ. Thêm backbone thứ tư cần adapter code, constraints, test và deploy, không phải thao tác dữ liệu thuần của Admin.

Catalog hiện có 27 endpoint. Phần đề xuất chính thêm 19 endpoint, thành 46 nếu làm đủ. Policy P2 thêm 3, thành 49. Các con số chỉ dùng kiểm tính nhất quán danh mục, không phải chỉ tiêu bắt buộc phải đạt.

9 Request mẫu và xử lý lỗi

Các JSON dưới đây là dữ liệu minh họa hợp đồng, không phải record thật hoặc credential cần đưa lên production.

Manager tạo User rồi gán vào Farm

```
POST /api/v1/manager/users
{
  "username": "farm_member_01",
  "email": "farm_member_01@example.com",
  "password": "StrongPass123!",
  "full_name": "Thanh vien 01"
}
```

```
POST /api/v1/farms/12/members
{"user_id": 45}
```

Service gán role=user và created_by từ Manager hiện tại. Client không gửi owner_id, added_by hoặc status để chiếm quyền. Chỉ các field trong schema của thao tác được phép cập nhật.

Kết quả Predict được rút gọn để đọc

```
{
  "scan_id": 82,
  "inference_mode": "standard",
  "validation_status": "accepted",
  "label": "Tomato__Early_blight",
  "confidence": 0.82,
  "models_requested": 2,
  "models_succeeded": 2,
  "farm_id": 12,
  "farm_assignment_status": "assigned",
  "agreement_status": "agreed"
}
```

Response thực còn Top-K, model_results, uncertainty, version và thông tin bệnh. Mẫu này không thay thế schema OpenAPI. FE không suy diễn đã lưu vào Farm nếu farm_assignment_status khác assigned.

Mã	Ý nghĩa	Quy tắc sử dụng
401 và 403	Token lỗi hoặc không đủ quyền	401 cho xác thực; 403 cho role hoặc phạm vi bị cấm như hiện tại
404 và 409	Không có đối tượng hoặc xung đột	409 cho duplicate mới, revision và review cạnh tranh; legacy có một số 400
413 415 422	Upload hoặc input không đúng	Trả lỗi rõ field; không trả stack trace
503 và 504	Model hoặc công suất không sẵn sàng; timeout	Không báo là ảnh bị bệnh hay không phải lá

10 Quy tắc CRUD và bàn giao API

Mỗi request ghi dữ liệu đi qua schema, xác thực, role, quyền trên đối tượng, điều kiện nghiệp vụ rồi mới ghi trong transaction. Những field audit như created_by, updated_by, added_by và reviewer_id luôn do server xác định. Enum ở API và CHECK trong DB phải được cập nhật cùng migration khi bổ sung miền giá trị.

Những quy tắc không được bỏ qua

- List phải lọc phạm vi ngay trong truy vấn, không lấy toàn bộ rồi lọc ở FE. Detail, ảnh, update và delete đều kiểm cùng ownership.
- Người sở hữu Farm không được tự động đọc Scan không gắn Farm, và không được bypass API owner-only.
- Member delete chỉ hủy phân công; không xóa User và không xóa lịch sử Scan đã gắn Farm.
- Approve phải bao gồm content, revision, status và audit trong một transaction; insert audit không được chứa password hoặc token.
- Scan chỉ được tạo bởi Predict; không mở sửa dự đoán để làm thay đổi lịch sử inference.
- Delete Scan cascade dữ liệu con và dọn cả ảnh gốc, explanation khi tính năng đó có. File lỗi cleanup cần retry hoặc quy trình dọn orphan hữu hạn.
- Activate không cho hai version cùng kiến trúc active; index là lớp chặn cuối cùng, service vẫn phải xử lý cạnh tranh.

Checklist bàn giao cho Frontend

FE tiếp tục dùng prefix /api/v1 và capability endpoint để dựng lựa chọn mode. Các API đề xuất phải có ví dụ thành công và lỗi trong OpenAPI trước khi FE tích hợp. API ảnh cần request Bearer và blob URL; refresh cookie dùng credentials và origin đã cho phép. Những thay đổi field bổ sung phải bảo đảm FE cũ bỏ qua field không biết được.

Trước khi đổi error 400 sang 409, đổi list thành envelope hoặc đổi Farm assignment sang strict, BE và FE chốt phiên bản tương thích. Không để tài liệu ghi một hành vi nhưng test tự động xác nhận hành vi khác.

Kịch bản nghiệm thu quyền

Manager A tạo hai User và một Farm; gán đúng User thành công. Manager B không thấy hoặc sửa được Farm đó và không gán được User của A. Một Managed User Predict vào Farm được giao rồi tự xem ảnh history. Admin gọi Scan của User qua /scans bị cấm nhưng được giám sát qua /admin/scans theo chính sách. Technician chỉ gửi proposal, không sửa Disease Info trực tiếp. Manager và Technician chỉ sinh explanation cho Scan của chính mình trong MVP.

Nguồn đối chiếu

Hiện trạng lấy từ backend/app/api/v1/endpoints; backend/app/api/deps.py; backend/app/schemas; backend/app/services/predict_service.py; prediction_workflow_service.py; scan_service.py; user_admin_service.py và image_storage_service.py. Các endpoint ĐỀ XUẤT trong tài liệu chưa có implementation tại ngày rà soát.

Nguồn về kiểm tra quyền ở từng đối tượng: OWASP API1 Broken Object Level Authorization.
<https://owasp.org/API-Security/editions/2023/en/Oxa1-broken-object-level-authorization/>